

Smart Agricultural Production Optimization Engine

Project Description:

The Smart Agricultural Production Optimization Engine (OptiCrop) is an AI and Machine Learning-based web application that helps farmers and agricultural stakeholders identify the most suitable crop based on soil nutrients and environmental conditions. The system predicts the best crop using parameters such as Nitrogen (N), Phosphorus (P), Potassium (K), temperature, humidity, pH value, and rainfall.

The application is developed using Python, Streamlit, Scikit-Learn, Pandas, and NumPy. A trained Machine Learning model analyzes the user inputs and provides accurate crop recommendations to improve productivity and optimize agricultural production. The user-friendly interface allows users to enter field conditions and instantly receive prediction results.

The project is deployed on Streamlit Community Cloud, making it accessible through any web browser without additional software installation. By providing data-driven crop recommendations, OptiCrop supports better farming decisions, efficient resource utilization, and sustainable agricultural practices.

Scenarios:

Scenario 1: Crop Recommendation

A farmer enters soil parameters such as Nitrogen (N), Phosphorus (P), Potassium (K), temperature, humidity, pH, and rainfall. The system analyzes the inputs using the trained machine learning model and recommends the most suitable crop for cultivation.

Scenario 2: First-Time User

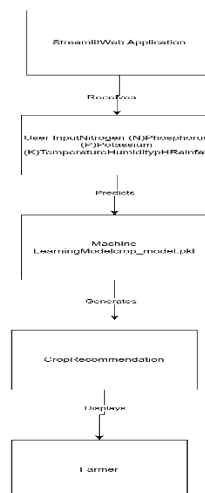
A new user accesses the deployed Streamlit application through the web browser. After entering the required agricultural parameters, the application instantly predicts the best crop and displays the recommendation in an easy-to-understand format without requiring any login.

Scenario 3: Decision Support for Better Farming

An agricultural advisor compares different soil and weather conditions by entering

multiple sets of values. The application provides crop recommendations for each case, helping farmers make informed decisions that improve productivity and reduce the risk of unsuitable crop selection.

Technical Architecture:



Description: Opti Crop Smart Agriculture is built using Streamlit as the web application framework and SciKit-Learn for machine learning. Users provide agricultural inputs including Nitrogen (N), Phosphorus (P), Potassium (K), temperature, humidity, pH, and rainfall through the Streamlit interface. The application processes these inputs using the trained `crop_model.pkl` model to predict the most suitable crop. The predicted crop recommendation is then displayed instantly to the user, providing a simple and efficient decision-support system for smart farming. The application is deployed on Streamlit Cloud, making it accessible through any web browser without requiring installation.

Pre-requisites:

- Python 3.10 or above
- Streamlit
- Pandas
- NumPy
- SciKit-Learn
- Jupyter
- Git (optional, for version control)
- Internet connection (for accessing the deployed application)

Project Workflow:

Module 1: Data Collection & Model Development

- Activity 1.1: Collect the crop recommendation dataset.

Activity 1.2: Preprocess the dataset by handling missing values and selecting required features.

- Activity 1.3: Train the Machine Learning model using SciKit-Learn

Activity 1.4.: Save the trained model as crop_model.pkl.

Module 2: Application Development

- Activity 2.1: Develop the user interface using Streamlit.
- Activity 2.2: Create input fields for Nitrogen, Phosphorus, Potassium, Temperature, Humidity, pH, and Rainfall.

Module 3: App.py Development

- **Activity 3.1:** Develop the main application logic in app.py, creating routes for user requests and integrating the machine learning model for crop recommendation

Milestone 4: Front end Development

- **Activity 4.1** Design a responsive user interface using Streamlit to collect soil nutrients, weather parameters, and environmental conditions
 - **Activity 4.2:** Display the recommended crop along with prediction details in a clean and user-friendly format..

Milestone 5: Deployment

- **Activity 5.1:** Test the application locally, verify model predictions, and ensure all dependencies are properly configured.
- **Activity 5.2:** Deploy the Crop Recommendation System using Streamlit Cloud (or Render/Heroku), making it accessible through a public URL.

Milestone 6: Conclusion

Milestone 1: Dataset Collection and Environment Set up

Activity 1.1: Install Python and Required Libraries

Before starting the development of the Crop Recommendation System, install Python and all required libraries. Create a virtual environment and install the dependencies such as Streamlit, Pandas, NumPy, SciKit-Learn, Joblib, and Pickle. These libraries are essential for running the machine learning model, processing user input, and building the web interface.

Steps:

Install Python (3.10 or above).

Create a virtual environment.

Install required packages using pip install -r requirement.txt or install Streamlit, Pandas, NumPy, SciKit-Learn, and Joblib individually.

Verify that all libraries are installed successfully.

Run the application locally using the Streamlit command

Activity 1.2: Research and Select the Appropriate Machine Learning Model

Use a supervised machine learning classification algorithm for crop prediction. Compare different algorithms such as Decision Tree, Random Forest, and Support Vector Machine based on prediction accuracy. Select the trained Random Forest model (crop_model.pkl) because it provides high accuracy and reliable crop recommendations.

Activity 1.3: Define the Architecture of the Application

Design the application architecture showing the interaction between the Streamlit front end, user inputs (Nitrogen, Phosphorus, Potassium, Temperature, Humidity, pH, Rainfall), the Machine Learning model (crop_model.pkl), and the predicted crop recommendation displayed to the farmer.

Activity 1.4.: Configure the project structure, organize the dataset and trained model, verify all dependencies, and test the Streamlit application locally before deployment.

- Create a Virtual Environment: Set up a virtual environment using venv to isolate project dependencies:

```
D:\project>python -m venv myenv
D:\project>"d:\project\venv\Scripts\activate"
(myenv) D:\project>pip install -r requirements.txt
```

- Install Flask: Use pip to install Flask:

```
(myenv) D:\project>pip install Flask==3.0.3
```

- Install Groq SDK: Install the Groq Python library to interact with the Groq API:

```
(myenv) D:\project>pip install groq==0.11.0
```

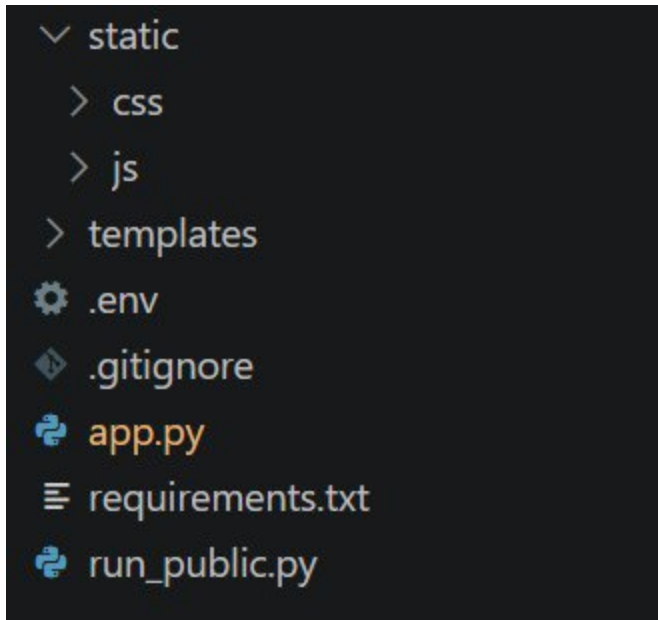
- Install python-dotenv: Install dotenv support for secure API key management:

```
(myenv) D:\project>pip install python-dotenv==1.0.1
```

- Configure the API Key: Create a .env file in the project root and add your Groq API key:

```
GROQ_API_KEY=your_groq_api_key_here
```

- Set Up Application Structure: Create the project directory structure including folders for templates, static files (CSS, JS), and main application files (app.py, requirements.txt, run_public.py).



Module 2: Core Functionalities Development

Activity 2.1: Develop Core Crop Recommendation Features

Implement the core functionalities of the Crop Recommendation System using a trained Machine Learning model.

User Input Module: Collect soil and environmental parameters such as Nitrogen (N), Phosphorus (P), Potassium (K), Temperature, Humidity, pH, and Rainfall from the user through the Streamlit interface.

Data Validation: Validate the entered values and preprocess the input before sending it to the prediction model.

Crop Prediction Module: Load the trained Machine Learning model (crop_model.pkl) and predict the most suitable crop based on the user inputs.

Recommendation Display: Display the predicted crop name clearly on the Streamlit web application along with a success message.

Activity 2.2: Integrate the Streamlit Application

Build Streamlit Interface: Develop a user-friendly web interface using Streamlit for collecting user inputs.

Model Integration: Connect the trained Machine Learning model with the Streamlit application for real-time prediction.

Testing: Verify that the application accepts valid inputs and displays accurate crop recommendations.

Performance Verification: Test the application with different datasets to ensure reliable prediction results.

Milestone 3: App.py Development

Milestone 3 focuses on developing and integrating the core application logic of the Crop Recommendation System. In this milestone, the Streamlit application is connected with the trained Machine Learning model to process user inputs, generate crop predictions, and display the recommended crop to the user.

Activity 3.1: Writing the Main Application Logic in app.py

Implement the Core Logic in app.py:

```
app.py
1  import streamlit as st
2  import pickle
3  import pandas as pd
4
5  # Load trained model
6  model = pickle.load(open("crop_model.pkl", "rb"))
7
```

Figure 3.1: Importing required libraries and loading the trained model.

Design the Streamlit interface to collect the following user inputs:

```
st.title(" 🌱 Smart Agricultural Production Optimization Engine")

temperature = st.number_input("Temperature (°C)", value=25)
rainfall = st.number_input("Rainfall (mm)", value=200)
humidity = st.number_input("Humidity (%)", value=80)
soil = st.selectbox("Soil Type", ["Loamy", "Black", "Sandy", "Clay"])
ph = st.number_input("Soil pH", value=6.5)
```

- Temperature (°C)
- Rainfall (mm)
- Humidity (%)
- Soil Type
- Soil pH

Convert the user inputs into the required format and pass them to the trained Machine Learning model for prediction.

```
soil_map = {
    "Loamy": 0,
    "Black": 1,
    "Sandy": 2,
    "Clay": 3
}
```

Display the predicted crop recommendation on the Streamlit web application.

```
if st.button("Predict Crop"):
```

Validate the application using different input values to ensure accurate prediction results.

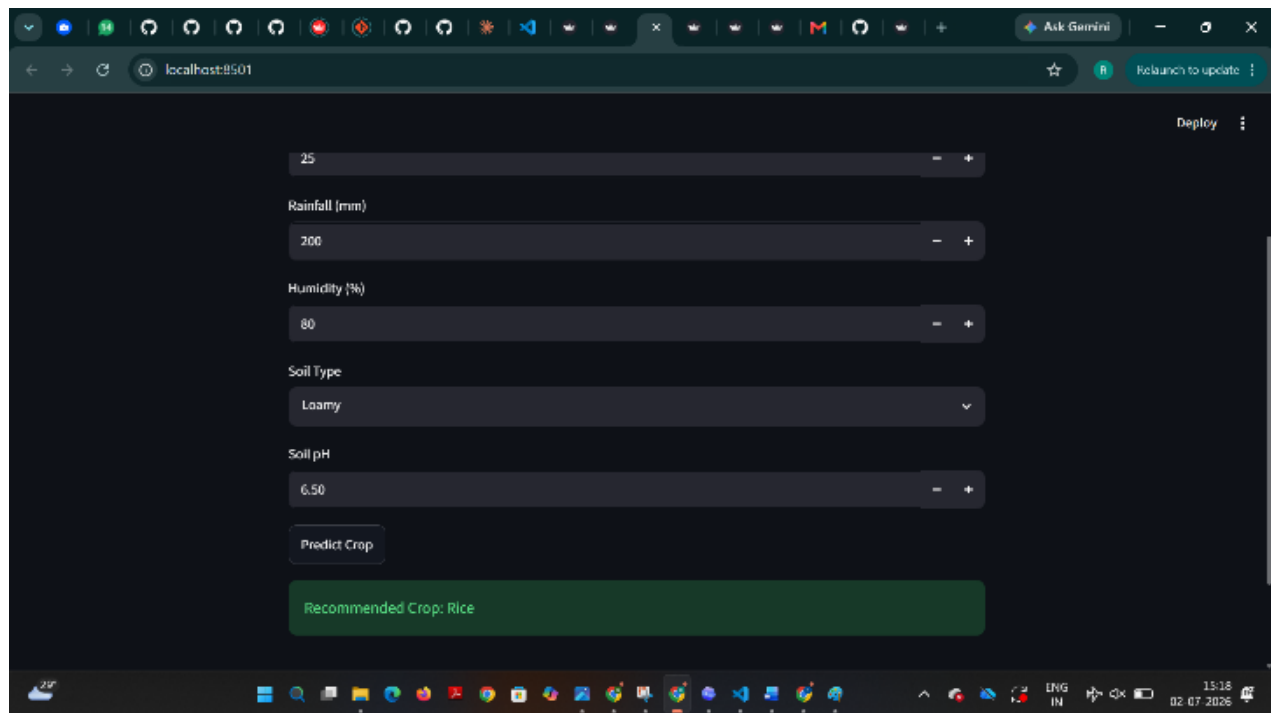
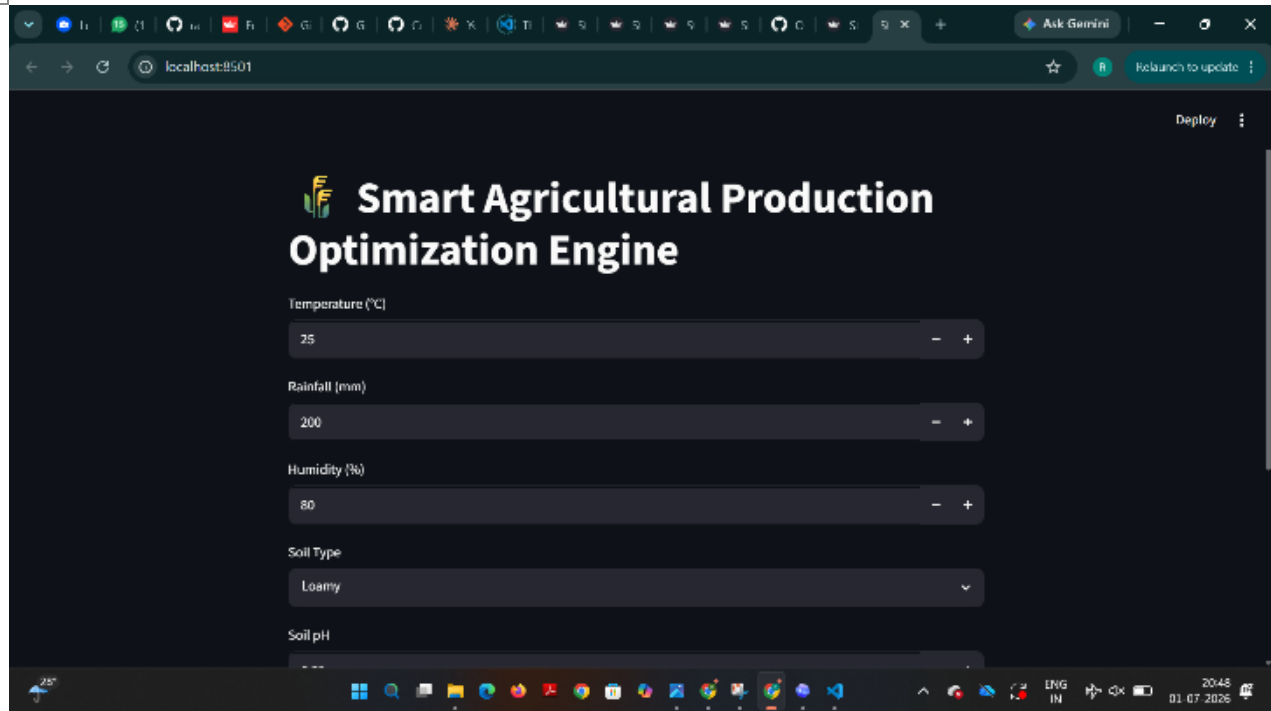


Figure 3.5: Final Streamlit application displaying the recommended crop (Rice).

Conclusion:-

The Smart Agricultural Production Optimization Engine was successfully developed using Python, Streamlit, and Machine Learning. The application accepts user inputs such as temperature, rainfall, humidity, soil type, and soil pH, and predicts the most suitable crop accurately. The system was tested with different input values, and the results were successfully displayed through the Streamlit web application.

